Homework 2: Convolutional Neural Networks

Math Section

Alexander Quispe Rojas
Universidad del Pacífico – MECA

Instructions: This homework covers the mathematical foundations of Convolutional Neural Networks (CNNs). You may discuss the problems with classmates but must write up your own solutions. Show all work for full credit. Total: **71 points**.

This homework is structured around four themes: (1) Convolution Fundamentals, (2) Pooling, (3) Architecture Analysis, (4) Backpropagation through Conv layers.

1 Convolution Fundamentals [29 Points]

Materials: Lectures 1, 2, 4 on perceptrons, backpropagation, and CNNs.

Problem 1 [5 Points]

Fully-Connected vs Convolutional Parameter Counting.
Consider an input image of size $32 \times 32 \times 3$ (e.g., a CIFAR-10 image).

(a) How many parameters (including biases) does a **fully-connected** layer with 128 output units have when the image is flattened?

(b) How many parameters does a **Conv2d** layer with 16 filters of size 5×5 have (with biases)?

(c) Compute the ratio of (a) to (b). What does this tell you about the parameter efficiency of CNNs?

(d) Explain in 2-3 sentences why this saving is achievable, in terms of *locality* and *weight sharing*.

Solution

(a) Fully-connected. A FC layer treats the input as a flat vector. We flatten the $32 \times 32 \times 3$ image:

$$\text{input dimension} = 32 \cdot 32 \cdot 3 = 3{,}072$$

For each of the 128 output units, we have 3,072 weights connecting it to every input pixel, plus 1 bias:

$$\text{params}_{\text{FC}} = \underbrace{128 \cdot 3{,}072}_{\text{weights}} + \underbrace{128}_{\text{biases}} = \boxed{393{,}344}$$

(b) Conv2d – understanding "16 filters".

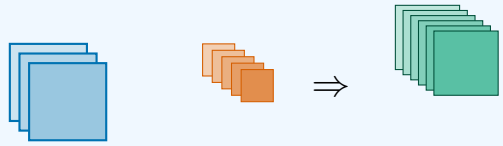
What is a filter? A filter (kernel) is a small matrix of learnable weights that slides across

the input, computing dot products at each position. Each filter is a **specialized feature detector**:

- One filter might learn to detect **vertical edges**.
- Another might detect **horizontal edges**.
- Another might detect a **green-red color transition**.
- Another might detect **circular blobs**, etc.

Why 16 filters? A single filter detects only one feature. To represent rich images, we need many feature detectors operating in *parallel*. Each filter produces its own output "feature map". The number 16 is a hyperparameter – common values are 32, 64, 128, 256.

Visualization of the layer:



Counting the parameters per filter:
Each filter has shape $\boxed{5 \times 5 \times 3}$ – the 3 *must* match the input channels (RGB), so the filter "sees" all colors at each spatial position.

$$\text{weights per filter} = 5 \cdot 5 \cdot 3 = 75$$

Each filter also has a single **bias** (a scalar added after the dot product). So:

$$\text{parameters per filter} = 75 + 1 = 76$$

Total for the layer (16 filters):

$$\text{params}_{\text{Conv}} = 16 \cdot 76 = \boxed{1{,}216}$$

In general, for a Conv2d layer with C_{out} filters of size $F \times F$ on input with C_{in} channels:

$$\boxed{\text{params} = C_{out} \cdot (F \cdot F \cdot C_{in} + 1)}$$

(c) Ratio.

$$\frac{\text{params}_{\text{FC}}}{\text{params}_{\text{Conv}}} = \frac{393{,}344}{1{,}216} \approx \boxed{323\times}$$

The fully-connected layer uses **323 times more parameters** for arguably less expressive power.

(d) Why is this saving possible? Two key inductive biases:

- **Locality**: each output depends only on a small 5×5 neighborhood of the input, not all 3,072 pixels. This matches the structure of natural images, where nearby pixels are correlated.

- **Weight sharing**: the same filter is reused at every spatial location. A vertical-edge detector should work *anywhere* in the image, not just in a specific corner. We only need to learn the filter *once* and apply it everywhere.

These are not just engineering tricks – they are **prior knowledge** (inductive biases) about images encoded into the architecture. The FC layer would have to *re-learn* from data that nearby pixels matter and that features are translation-invariant. CNNs build these priors in for free.

~~Problem 2 [8 Points]~~

2D Convolution by Hand.
Consider the 5×5 input image $\mathbf{X}$ and 3×3 kernel $\mathbf{K}$:

$$\mathbf{X} = \begin{pmatrix} 1 & 2 & 3 & 0 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 2 & 1 & 0 & 1 & 2 \\ 1 & 0 & 2 & 3 & 0 \\ 0 & 1 & 1 & 2 & 1 \end{pmatrix}, \quad \mathbf{K} = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

This is a vertical edge detector (similar to the Sobel filter).

(a) Compute the output $\mathbf{Y} = \mathbf{X} * \mathbf{K}$ with stride 1 and no padding. Show the dot product for each output position.

(b) What is the shape of $\mathbf{Y}$? Verify with the formula $W_{\text{out}} = \lfloor (W_{\text{in}} + 2P - F)/S \rfloor + 1$.

(c) Now compute $\mathbf{Y}$ with stride 2 and padding 1 (zero-padding). What is the new shape?

Solution

(a) Stride 1, no padding. Output size: $(5-3)/1+1 = 3$, so $\mathbf{Y}$ is 3×3.
For each output position (i, j), $Y_{i,j} = \sum_{u=0}^{2} \sum_{v=0}^{2} X_{i+u,j+v} K_{u,v}$.
Position (0,0): window over rows 0-2, cols 0-2:

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \\ 2 & 1 & 0 \end{pmatrix} \odot \mathbf{K} = (1+0+2) + (0+0+0) + (-3-2-0) = 3+0-5 = -2$$

Position (0,1): window over rows 0-2, cols 1-3 has left column $(2, 1, 1)$ and right column $(0, 3, 1)$:

$$(2+1+1) - (0+3+1) = 4-4 = 0$$

Position (0,2): window over rows 0-2, cols 2-4:

$$(3+2+0) + (0+0+0) + (-1-1-2) = 5+0-4 = 1$$

Position (1,0): rows 1-3, cols 0-2: $(0+2+1) + (0) + (-2-0-2) = 3-4 = -1$.
Position (1,1): rows 1-3, cols 1-3: $(1+1+0) + (0) + (-3-1-3) = 2-7 = -5$.
Position (1,2): rows 1-3, cols 2-4: $(2+0+2) + (0) + (-1-2-0) = 4-3 = 1$.
Position (2,0): rows 2-4, cols 0-2: $(2+1+0) + (0) + (-0-2-1) = 3-3 = 0$.
Position (2,1): rows 2-4, cols 1-3: $(1+0+1) + (0) + (-1-3-2) = 2-6 = -4$.
Position (2,2): rows 2-4, cols 2-4: left $(0, 2, 1)$, right $(2, 0, 1)$. $3-3 = 0$.

$$\mathbf{Y} = \begin{pmatrix} -2 & 0 & 1 \\ -1 & -5 & 1 \\ 0 & -4 & 0 \end{pmatrix}$$

(b) Shape: $W_{\text{out}} = \lfloor (5+0-3)/1 \rfloor + 1 = 3$. So $\mathbf{Y} \in \mathbb{R}^{3 \times 3}$. ✓
(c) Stride 2, padding 1. Output size: $\lfloor (5+2-3)/2 \rfloor + 1 = 3$. So $\mathbf{Y} \in \mathbb{R}^{3 \times 3}$ (still 3×3 but covering different positions).

~~Problem 3 [4 Points]~~

Output Size Formula (Multi-channel).
Consider a Conv2d layer with input shape $H \times W \times C_{in}$ and:

- Filter size: $F \times F$
- Number of filters: C_{out}
- Stride: S
- Padding: P

(a) Derive the output shape $H' \times W' \times C'$.

(b) Derive the number of parameters in this layer (with biases).

(c) Apply your formulas to: input $224 \times 224 \times 3$, $F = 7$, $C_{out} = 64$, $S = 2$, $P = 3$. Give output shape and parameter count.

Solution

(a) Output shape:

$$H' = \left\lfloor \frac{H + 2P - F}{S} \right\rfloor + 1, \qquad W' = \left\lfloor \frac{W + 2P - F}{S} \right\rfloor + 1, \qquad C' = C_{out}$$

(b) Parameter count:
Each filter has size $F \times F \times C_{in}$ (must match all input channels). With C_{out} filters and one bias per filter:

$$\text{params} = F \cdot F \cdot C_{in} \cdot C_{out} + C_{out}$$

(c) Application to ResNet first layer:

$$H' = W' = \lfloor (224 + 6 - 7)/2 \rfloor + 1 = \lfloor 223/2 \rfloor + 1 = 111 + 1 = \boxed{112}$$

Output shape: $\boxed{112 \times 112 \times 64}$.
Parameters: $7 \cdot 7 \cdot 3 \cdot 64 + 64 = 9{,}408 + 64 = \boxed{9{,}472}$.

~~Problem 4 [4 Points]~~

Hand-Designed Edge Detector.

(a) Construct a 3×3 kernel $\mathbf{K}_v$ that detects **vertical** edges (i.e., produces a strong activation when there is a sharp horizontal change in pixel intensity).

(b) Apply your kernel to the following 4×4 image with a vertical edge:

$$\mathbf{X} = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

Compute the output (no padding, stride 1).

(c) Modify your kernel to detect horizontal edges instead. Show the kernel and compute the output on the same image.

Solution

(a) Vertical edge detector:

$$\mathbf{K}_v = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

This is the (positive) Sobel-like vertical edge filter. It computes *(sum of right column) - (sum of left column)*, which is large when there's a horizontal change in pixel value.

(b) Apply to X. Output size: $4 - 3 + 1 = 2$, so $\mathbf{Y}$ is 2×2.

Position (0,0): columns of $\mathbf{X}[0:3, 0:3]$ are $(0,0,0), (0,0,0), (1,1,1)$. Output $= 3 - 0 = 3$.

Position (0,1): columns of $\mathbf{X}[0:3, 1:4]$ are $(0,0,0), (1,1,1), (1,1,1)$. Output $= 3 - 0 = 3$.

Position (1,0): same as (0,0) since rows are constant: 3. **Position (1,1):** same as (0,1): 3.

$$\mathbf{Y} = \begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$$

The detector activates uniformly along the vertical edge, as expected.

(c) Horizontal edge detector:

$$\mathbf{K}_h = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

Applied to $\mathbf{X}$ (which has *no* horizontal edges), output should be near 0:

Position (0,0): $\mathbf{X}[0:3, 0:3]$, all rows have same values, so (top row sum) $-$ (bottom row sum) $= 0$. Similarly for all positions.

$$\mathbf{Y} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

The horizontal edge detector correctly produces zero everywhere on this image (which has no horizontal edges).

~~Problem 5 [4 Points]~~

Padding Modes Comparison.

(a) Define the three common padding modes: *valid*, *same*, and *full*.

(b) Compute the output size for each mode given an input of size W and filter of size F (assume stride 1).

(c) Show that "same" padding requires $P = (F-1)/2$ for stride 1 and odd F. What happens for even F?

(d) Why is "same" padding particularly useful when stacking many conv layers?

Solution

(a) Definitions:

- *Valid* ($P = 0$): no padding, only "valid" positions used.
- *Same*: pad enough so $W_{\text{out}} = W_{\text{in}}$ (with stride 1).
- *Full*: pad enough to use every position where the kernel and input overlap by at least one element.

(b) Output sizes (stride 1):

- Valid: $W' = W - F + 1$.
- Same: $W' = W$.
- Full: $W' = W + F - 1$.

(c) "Same" padding requirement. From $W' = W \Leftrightarrow W + 2P - F + 1 = W \Leftrightarrow P = (F-1)/2$. This requires F to be **odd** for P to be an integer. For even F, asymmetric padding is needed.

(d) Why useful? When stacking L conv layers with valid padding, output shrinks by $L(F-1)$. Without padding, depth is limited by input size. Same padding lets you stack arbitrarily many layers without spatial collapse.

~~Problem 6 [4 Points]~~

Receptive Field Through a Sample Network.

Compute the receptive field at each layer of the following network applied to a 28×28 image. Use the recurrences:

$$j_l = j_{l-1} \cdot S_l, \qquad \mathrm{RF}_l = \mathrm{RF}_{l-1} + (F_l - 1) \cdot j_{l-1}$$

with initial conditions $j_0 = 1, \mathrm{RF}_0 = 1$.

l	Operation	F_l	S_l	Output size
0	Input	—	—	28×28
1	Conv 5×5	5	1	?
2	MaxPool 2×2	2	2	?
3	Conv 5×5	5	1	?
4	MaxPool 2×2	2	2	?
5	Conv 3×3	3	1	?

Compute output size, j_l, and RF_l at each layer.

Solution

l	Operation	Output size	$j_l = j_{l-1} S_l$	$\mathrm{RF}_l = \mathrm{RF}_{l-1} + (F_l - 1) j_{l-1}$
0	Input	28×28	1	1
1	Conv 5x5 (no pad)	24×24	$1 \cdot 1 = 1$	$1 + 4 \cdot 1 = \boxed{5}$
2	MaxPool 2x2 s=2	12×12	$1 \cdot 2 = \mathbf{2}$	$5 + 1 \cdot 1 = \boxed{6}$
3	Conv 5x5 (no pad)	8×8	$2 \cdot 1 = 2$	$6 + 4 \cdot 2 = \boxed{14}$
4	MaxPool 2x2 s=2	4×4	$2 \cdot 2 = \mathbf{4}$	$14 + 1 \cdot 2 = \boxed{16}$
5	Conv 3x3 (no pad)	2×2	$4 \cdot 1 = 4$	$16 + 2 \cdot 4 = \boxed{24}$

Final receptive field: a single neuron at layer 5 sees a 24×24 patch of the input, almost the entire 28×28 image.

Observation: pooling layers *double* the jump factor j_l, which then *multiplies* the receptive field increment of subsequent layers. This is why pooling accelerates RF growth.

2 Pooling [8 Points]

Materials: Lecture 4 on CNNs.

Problem 7 [4 Points]

Max vs Average Pooling.
Consider the 4×4 feature map:

$$\mathbf{X} = \begin{pmatrix} 2 & 5 & 1 & 3 \\ 4 & 1 & 8 & 0 \\ 7 & 2 & 1 & 4 \\ 3 & 0 & 6 & 9 \end{pmatrix}$$

(a) Apply 2×2 **max pooling** with stride 2.

(b) Apply 2×2 **average pooling** with stride 2.

(c) Discuss: when would you prefer max over average pooling?

Solution

(a) Max pool: divide into four 2×2 regions, take max of each:
Top-left: $\max(2, 5, 4, 1) = 5$. Top-right: $\max(1, 3, 8, 0) = 8$.
Bot-left: $\max(7, 2, 3, 0) = 7$. Bot-right: $\max(1, 4, 6, 9) = 9$.

$$\mathbf{Y}_{\text{max}} = \begin{pmatrix} 5 & 8 \\ 7 & 9 \end{pmatrix}$$

(b) Average pool:
Top-left: $(2 + 5 + 4 + 1)/4 = 3$. Top-right: $(1 + 3 + 8 + 0)/4 = 3$.
Bot-left: $(7 + 2 + 3 + 0)/4 = 3$. Bot-right: $(1 + 4 + 6 + 9)/4 = 5$.

$$\mathbf{Y}_{\text{avg}} = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}$$

(c) When to use which?

- **Max pool**: preserves the strongest activation → good for detection tasks ("is this feature present anywhere in the region?"). Standard choice in classic CNNs.
- **Average pool**: smoother, retains more information. Used in modern architectures for the final aggregation (e.g., global average pooling in ResNet).

~~Problem 8 [4 Points]~~

Pooling Effect on Receptive Field.
Consider a network with the following layers (all using 3×3 kernels, stride 1, no padding):

$$\text{Conv} \to \text{Conv} \to \text{MaxPool}(2 \times 2, \text{ stride } 2) \to \text{Conv} \to \text{Conv}$$

(a) What is the receptive field after the first two conv layers?

(b) What is the jump factor j after the maxpool layer?

(c) What is the receptive field after the final conv layer?

Solution

Apply the recurrences with $j_0 = 1, \text{RF}_0 = 1$:

l	Operation	j_l	RF_l
1	Conv 3x3	1	$1 + 2 \cdot 1 = 3$
2	Conv 3x3	1	$3 + 2 \cdot 1 = 5$
3	MaxPool 2x2 s=2	**2**	$5 + 1 \cdot 1 = 6$
4	Conv 3x3	2	$6 + 2 \cdot 2 = 10$
5	Conv 3x3	2	$10 + 2 \cdot 2 = \boxed{14}$

(a) After 2 conv layers: $\text{RF} = 5$. **(b)** Jump after maxpool: $j = 2$. **(c)** Final RF: 14×14 on the input.
Insight: the pooling layer added only 1 to RF directly, but it *doubled* the jump, so each subsequent conv layer adds $2(F-1) = 4$ to RF instead of $F - 1 = 2$. Pooling *accelerates* RF growth.

3 Architecture Analysis [14 Points]

Materials: Lecture 4: classic CNN architectures (LeNet, VGG).

Problem 9 [7 Points]

LeNet-5 Parameter Count.

The original LeNet-5 (LeCun et al., 1998) has the following architecture for 32×32 grayscale input:

Layer	Operation	Output size	Parameters
1	Conv 5×5, 6 filters	$28 \times 28 \times 6$	?
2	AvgPool 2×2, stride 2	$14 \times 14 \times 6$	0
3	Conv 5×5, 16 filters	$10 \times 10 \times 16$	?
4	AvgPool 2×2, stride 2	$5 \times 5 \times 16$	0
5	FC $\rightarrow$ 120	120	?
6	FC $\rightarrow$ 84	84	?
7	FC $\rightarrow$ 10	10	?

(a) Compute the parameter count for each layer (with biases).

(b) What is the total parameter count?

(c) Compare with an MLP that has 2 hidden layers of 300 units each. Which has more parameters?

Solution

(a) Layer-by-layer:

- Layer 1: $5 \cdot 5 \cdot 1 \cdot 6 + 6 = 156$.
- Layer 3: $5 \cdot 5 \cdot 6 \cdot 16 + 16 = 2{,}416$.
- Layer 5 (FC $400 \rightarrow 120$): $400 \cdot 120 + 120 = 48{,}120$.
- Layer 6 (FC $120 \rightarrow 84$): $120 \cdot 84 + 84 = 10{,}164$.
- Layer 7 (FC $84 \rightarrow 10$): $84 \cdot 10 + 10 = 850$.

(b) Total: $156 + 2{,}416 + 48{,}120 + 10{,}164 + 850 = \boxed{61{,}706}$ parameters.

(c) MLP comparison:

For input $1024 \rightarrow 300 \rightarrow 300 \rightarrow 10$:

$$1024 \cdot 300 + 300 + 300 \cdot 300 + 300 + 300 \cdot 10 + 10 = 307{,}510 + 90{,}300 + 3{,}010 = 400{,}820$$

The MLP has **6.5x more parameters** than LeNet-5, despite LeNet achieving better accuracy. Inductive biases (locality + weight sharing) make CNNs far more parameter-efficient. Note: most LeNet parameters are in the FC layers, not the conv layers. This motivated later architectures (e.g., ResNet) to use *global average pooling* instead of FC.

~~Problem 10 [7 Points]~~

Why 3×3 Filters Dominate.

In modern CNNs (VGG, ResNet, etc.), the 3×3 filter has become standard. Compare two architectures with the same effective receptive field of 5×5:

- **Architecture A**: One Conv 5×5 layer with C input and C output channels.
- **Architecture B**: Two stacked Conv 3×3 layers (each $C \to C$).

(a) Compute the receptive field of each architecture (verify both are 5×5).

(b) Compute the parameter count of each (ignore biases for simplicity).

(c) Count the number of nonlinearities (ReLU layers) each can apply.

(d) Discuss: why is Architecture B preferred?

Solution

(a) Receptive field:

- Architecture A: RF $= 5$ (a 5×5 filter sees a 5×5 patch).
- Architecture B: applying the recurrence: $\text{RF}_1 = 1 + (3-1) \cdot 1 = 3$, then $\text{RF}_2 = 3 + (3-1) \cdot 1 = 5$.

Both: RF $= 5$. ✓

(b) Parameter count (no biases):

- Architecture A: $5 \cdot 5 \cdot C \cdot C = 25C^2$.
- Architecture B: $2 \cdot (3 \cdot 3 \cdot C \cdot C) = 18C^2$.

B uses $18/25 = 72\%$ of A's parameters, a $\sim 28\%$ saving.

(c) Nonlinearities:

- Architecture A: 1 ReLU.
- Architecture B: 2 ReLUs (one between each conv).

B has **twice the non-linear expressivity**.

(d) Why prefer B?

Architecture B has:

1. **Fewer parameters** (28% less) for the same RF.
2. **More nonlinearities** (twice as many ReLUs) $\to$ more expressive.
3. **Easier optimization** (smaller filters $\to$ more uniform gradient distributions).

Generalization: stacking n 3×3 convs gives RF $= 2n + 1$ with $9nC^2$ parameters, vs $(2n+1)^2C^2$ for a single big conv. The savings grow with n.

Historical: this insight was a key contribution of VGG (2014).

4 Backpropagation Through Conv Layers [20 Points]

Materials: Lecture 2 on backpropagation, Lecture 4 on CNNs.

Problem 11 [10 Points]

Gradient w.r.t. the Convolution Kernel.

Consider a 1D convolution (single channel) with input $\mathbf{x} \in \mathbb{R}^n$, kernel $\mathbf{K} \in \mathbb{R}^k$, and output $\mathbf{y} \in \mathbb{R}^{n-k+1}$:

$$y_i = \sum_{u=0}^{k-1} x_{i+u} K_u, \qquad i = 0, 1, \ldots, n-k$$

Let $\mathcal{L}$ be a downstream loss. Define $\delta_i \equiv \partial \mathcal{L} / \partial y_i$ (the upstream gradient).

(a) Derive a formula for $\partial \mathcal{L} / \partial K_u$.

(b) Derive a formula for $\partial \mathcal{L} / \partial x_j$.

(c) Show that the gradient w.r.t. the kernel is itself a **convolution** between the input $\mathbf{x}$ and the upstream gradient $\boldsymbol{\delta}$.

(d) Show that the gradient w.r.t. the input is a **full convolution** between $\boldsymbol{\delta}$ and the **flipped** kernel.

Solution

(a) Gradient w.r.t. kernel.
Using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial K_u} = \sum_{i=0}^{n-k} \frac{\partial \mathcal{L}}{\partial y_i} \frac{\partial y_i}{\partial K_u} = \sum_{i=0}^{n-k} \delta_i \cdot x_{i+u}$$

(b) Gradient w.r.t. input.
Each x_j contributes to outputs y_i where $i + u = j$, i.e., $i = j - u$, for $u = 0, \ldots, k-1$:

$$\frac{\partial \mathcal{L}}{\partial x_j} = \sum_{u=0}^{k-1} \frac{\partial \mathcal{L}}{\partial y_{j-u}} \frac{\partial y_{j-u}}{\partial x_j} = \sum_{u=0}^{k-1} \delta_{j-u} \cdot K_u$$

(with $\delta_i = 0$ outside its valid range).

(c) Kernel gradient as a convolution.
Rewrite (a):

$$\frac{\partial \mathcal{L}}{\partial K_u} = \sum_i \delta_i \, x_{i+u} = (\boldsymbol{\delta} * \mathbf{x})[u]$$

This is the cross-correlation of $\boldsymbol{\delta}$ with $\mathbf{x}$ (or equivalently, the convolution of $\mathbf{x}$ with $\boldsymbol{\delta}$). ■

(d) Input gradient as a flipped convolution.
Let $\tilde{\mathbf{K}}$ be the flipped kernel: $\tilde{K}_u = K_{k-1-u}$. Then:

$$\frac{\partial \mathcal{L}}{\partial x_j} = \sum_u \delta_{j-u} K_u = \sum_{u'} \delta_{j-(k-1-u')} K_{k-1-u'} = \sum_{u'} \delta_{j-k+1+u'} \tilde{K}_{u'}$$

which is a (full) convolution of $\boldsymbol{\delta}$ with $\tilde{\mathbf{K}}$. ■

Implementation note: this is why PyTorch's backward pass for `Conv2d` is implemented as another conv operation. The convolution is its own "adjoint" (up to flipping).

~~Problem 12 [5 Points]~~

Gradient Through Max Pooling.
Consider a 2×2 max pooling layer with stride 2. Let the output at position (i, j) be:

$$y_{i,j} = \max_{(u,v) \in W_{i,j}} x_{u,v}$$

where $W_{i,j}$ is the 2×2 window for output (i, j).

(a) Derive $\partial \mathcal{L} / \partial x_{u,v}$ in terms of the upstream gradient $\delta_{i,j} = \partial \mathcal{L} / \partial y_{i,j}$.

(b) Apply your formula: given input

$$\mathbf{X} = \begin{pmatrix} 1 & 3 & 2 & 0 \\ 5 & 4 & 1 & 6 \\ 0 & 2 & 7 & 3 \\ 1 & 1 & 5 & 8 \end{pmatrix}, \quad \boldsymbol{\delta} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Compute $\partial \mathcal{L} / \partial \mathbf{X}$ entry-by-entry.

(c) Why is the gradient sparse?

Solution

(a) Formula. The max function is differentiable except at ties. Almost everywhere:

$$\frac{\partial y_{i,j}}{\partial x_{u,v}} = \begin{cases} 1 & \text{if } (u,v) = \arg\max_{W_{i,j}} \mathbf{X} \\ 0 & \text{otherwise} \end{cases}$$

Therefore:

$$\frac{\partial \mathcal{L}}{\partial x_{u,v}} = \begin{cases} \delta_{i,j} & \text{if } (u,v) \text{ is the argmax of window } W_{i,j} \\ 0 & \text{otherwise} \end{cases}$$

(b) Application. Argmax positions for each window:

- Top-left window $\mathbf{X}[0:2, 0:2] = \begin{pmatrix} 1 & 3 \\ 5 & 4 \end{pmatrix}$, max at $(1,0) = 5$. So $\delta_{0,0} = 1$ flows to position $(1,0)$.
- Top-right window $\mathbf{X}[0:2, 2:4] = \begin{pmatrix} 2 & 0 \\ 1 & 6 \end{pmatrix}$, max at $(1,3) = 6$. $\delta_{0,1} = 2$ flows to $(1,3)$.
- Bot-left $\mathbf{X}[2:4, 0:2] = \begin{pmatrix} 0 & 2 \\ 1 & 1 \end{pmatrix}$, max at $(2,1) = 2$. $\delta_{1,0} = 3$ flows to $(2,1)$.
- Bot-right $\mathbf{X}[2:4, 2:4] = \begin{pmatrix} 7 & 3 \\ 5 & 8 \end{pmatrix}$, max at $(3,3) = 8$. $\delta_{1,1} = 4$ flows to $(3,3)$.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 2 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 4 \end{pmatrix}$$

(c) Sparsity. Only one position per pooling window receives gradient (the argmax). All other positions receive zero. **Sparse** backward pass: gradient flow is concentrated.
This is in contrast to conv layers, where the gradient is dense (every input position receives gradient from every output position it touches).

~~Problem 13 [5 Points]~~

Gradient Through Average Pooling.
Repeat the previous problem for 2×2 average pooling with stride 2:

$$y_{i,j} = \frac{1}{4} \sum_{(u,v) \in W_{i,j}} x_{u,v}$$

(a) Derive $\partial \mathcal{L} / \partial x_{u,v}$ in terms of $\delta_{i,j}$.

(b) Apply to the same input and upstream gradient as before. Compute $\partial \mathcal{L} / \partial \mathbf{X}$.

(c) Compare with max pool: which is more "democratic"?

Solution

(a) Formula.

$$\frac{\partial y_{i,j}}{\partial x_{u,v}} = \begin{cases} 1/4 & \text{if } (u,v) \in W_{i,j} \\ 0 & \text{otherwise} \end{cases}$$

So:

$$\frac{\partial \mathcal{L}}{\partial x_{u,v}} = \frac{1}{4} \delta_{i,j} \quad \text{where } W_{i,j} \text{ contains } (u,v)$$

(b) Application.
Each $\delta_{i,j}$ is divided by 4 and broadcast to its 2×2 window:

- $\delta_{0,0}/4 = 0.25$ to all of top-left window.
- $\delta_{0,1}/4 = 0.5$ to all of top-right window.
- $\delta_{1,0}/4 = 0.75$ to all of bot-left window.
- $\delta_{1,1}/4 = 1.0$ to all of bot-right window.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \begin{pmatrix} 0.25 & 0.25 & 0.5 & 0.5 \\ 0.25 & 0.25 & 0.5 & 0.5 \\ 0.75 & 0.75 & 1.0 & 1.0 \\ 0.75 & 0.75 & 1.0 & 1.0 \end{pmatrix}$$

(c) Comparison.
Average pool is "democratic": every position in each window receives an equal share (1/4) of the gradient. Max pool is "winner-takes-all": only the argmax receives gradient.
Implication: max pool can lead to "dead" positions (some never receive gradient), while average pool ensures all positions are updated. But max pool tends to encourage sparser, more decisive features.

Summary of Topics Covered

This homework covered:

- **Fundamentals:** parameter counting, convolution by hand, padding modes, edge detectors.

- **Pooling:** max vs avg pooling, effect on receptive field.
- **Architecture:** receptive field, LeNet parameter accounting, why 3×3 filters dominate.
- **Backpropagation:** gradient w.r.t. kernel and input, max/avg pool gradients.

Recommended reading:

- Goodfellow et al., *Deep Learning*, Chapter 9 (CNNs).
- He et al. (2015), *Deep Residual Learning for Image Recognition.*
- LeCun et al. (1998), *Gradient-Based Learning Applied to Document Recognition* (LeNet-5).
- Krizhevsky, Sutskever, Hinton (2012), *ImageNet Classification with Deep Convolutional Neural Networks* (AlexNet).